

Busca Binária — Parte I

Universidade Estadual do Sudoeste da Bahia - UESB

2026.1

Motivação: Counting Haybales

Enunciado: O fazendeiro John possui N fardos de feno ($1 \leq N \leq 10^5$) em vários pontos ao longo de sua estrada. Ajude-o a responder Q perguntas ($1 \leq Q \leq 10^5$), cada uma solicitando o número de fardos dentro de um intervalo específico.

Entrada:

```
4 6
3 2 7 5
2 3
2 4
2 5
2 7
4 6
8 10
```

Saída:

```
2
2
3
4
1
0
```

O que é Busca Binária?

Algoritmo de busca em estruturas de dados **ordenadas**.

Funcionamento — para buscar um elemento x em uma estrutura indexada de $[1, N]$:

- ▶ Caso o intervalo atual de busca possua apenas 1 termo, finaliza-se o algoritmo comparando-o com x .
- ▶ Caso contrário, compara-se o elemento central ($N/2$) com x :
 - ▶ Se for **igual**, então finaliza-se a busca.
 - ▶ Se for **menor**, repete-se a busca para o intervalo superior ($]N/2, N]$).
 - ▶ Se for **maior**, repete-se a busca para o intervalo inferior ($[1, N/2[$).

Busca Binária — Visualização

Search for 47

0	4	7	10	14	23	45	47	53
---	---	---	----	----	----	----	----	----

Busca Binária — Visualização

Search for 47



0	4	7	10	14	23	45	47	53
---	---	---	----	----	----	----	----	----

Busca Binária — Visualização

Search for 47

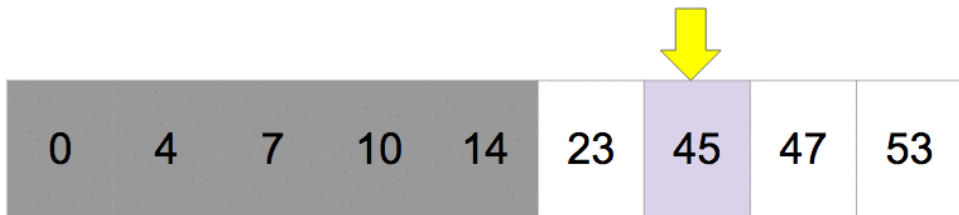


0	4	7	10	14	23	45	47	53
---	---	---	----	----	----	----	----	----

$14 < 47$

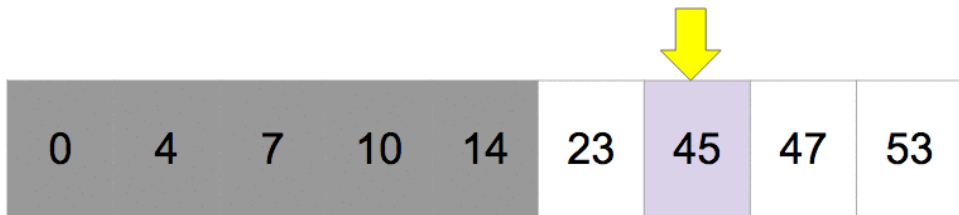
Busca Binária — Visualização

Search for 47



Busca Binária — Visualização

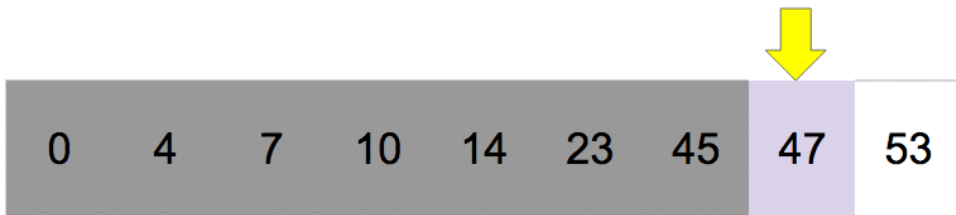
Search for 47



$45 < 47$

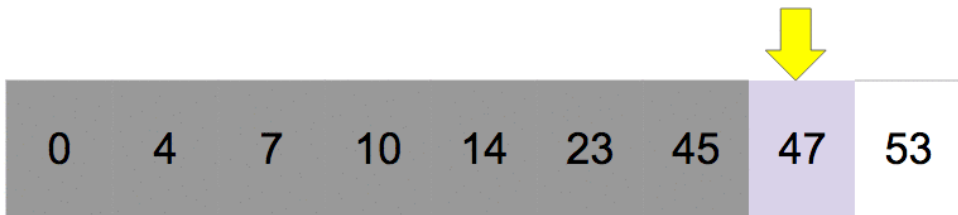
Busca Binária — Visualização

Search for 47



Busca Binária — Visualização

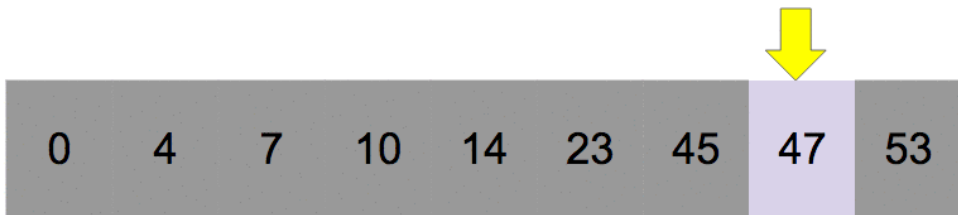
Search for 47



$47 = 47$

Busca Binária — Visualização

Search for 47



$47 = 47$

47 is in
the list!

Complexidade da Busca Binária

Seja z o número de operações realizadas pelo algoritmo e n a quantidade de termos do intervalo. A cada iteração o intervalo é dividido ao meio, logo:

$$\frac{n}{2^z} = 1 \implies n = 2^z \implies z = \log_2(n)$$

Portanto, a complexidade do algoritmo no pior caso é:

$$\mathbf{O(\log n)}$$

Busca Binária — Código (Estrutura de Acesso Aleatório)

```
int a = 0, b = n - 1;
while (a <= b) {
    int m = a + (b - a) / 2; // equivalente a (a+b)/2, porem
                             evita integer overflow
    if (array[m] == x) {
        return m;
    }
    if (array[m] > x) b = m - 1;
    else             a = m + 1;
}
return -1; // elemento nao encontrado
```

Pré-requisitos da Busca Binária

- ▶ A estrutura de dados deve estar **ordenada**.
- ▶ Buscar um elemento uma única vez em um vetor não ordenado — $O(n)$ — é **mais eficiente** do que ordená-lo e depois aplicar a busca binária — $O(n \log n)$.
- ▶ Portanto, a busca binária é vantajosa quando são necessárias **múltiplas consultas** sobre a mesma estrutura.

Implementação em C++ — Vetores

- ▶ `lower_bound(v.begin(), v.end(), x)`
Retorna um iterador para o **primeiro elemento igual ou maior** que `x`.
- ▶ `upper_bound(v.begin(), v.end(), x)`
Retorna um iterador para o **primeiro elemento estritamente maior** que `x`.
- ▶ **Obs.:** o vetor deve estar necessariamente **ordenado**.
- ▶ **Obs.:** se nenhum termo corresponder à condição, o iterador `end()` é retornado — ele **não aponta para nenhum elemento** do vetor.

Implementação em C++ — Vetores (Exemplo)

```
vector<int> v = {1, 3, 3, 5, 7, 9};

auto lb = lower_bound(v.begin(), v.end(), 3); // v[1]
auto ub = upper_bound(v.begin(), v.end(), 3); // v[3]
cout << *lb << "\n"; // 3
cout << *ub << "\n"; // 5

auto lb2 = lower_bound(v.begin(), v.end(), 4); // v[3]
auto ub2 = upper_bound(v.begin(), v.end(), 4); // v[3]
cout << *lb2 << "\n"; // 5
cout << *ub2 << "\n"; // 5
```


Implementação em C++ — Sets, Multisets e Maps

- ▶ Essas estruturas possuem sua **própria função** de `lower_bound` / `upper_bound` como método membro.
- ▶ Segundo o `cppreference.com`: *"std::map, std::set, and std::multiset iterators are not random access, and so their member lower_bound functions should be preferred."*

Obs.: `multiset` é uma estrutura semelhante ao `set`, porém permite a inserção de múltiplos termos iguais.

Implementação em C++ — Sets, Multisets e Maps (Exemplo)

```
multiset<int> m = {5, 2, 8, 2, 5};  
// m -> {2, 2, 5, 5, 8}  
  
cout << *m.lower_bound(5) << endl; // 5 (primeiro >= 5)  
cout << *m.upper_bound(5) << endl; // 8 (primeiro > 5)  
cout << *m.lower_bound(1) << endl; // 2 (primeiro >= 1)  
cout << *m.upper_bound(1) << endl; // 2 (primeiro > 1)
```

Quando usar `lower_bound` ou `upper_bound`?

Verificar se um elemento está no array:

Quando usar `lower_bound` ou `upper_bound`?

Verificar se um elemento está no array:

```
int k = lower_bound(v.begin(), v.end(), x) - v.begin();  
if (k < n && array[k] == x) {  
    // o array possui o elemento x na posicao k  
}
```

Obs.: a subtração entre iteradores retorna a distância (em posições) entre os dois ponteiros.
Funciona apenas para iteradores de vectors

Quando usar `lower_bound` ou `upper_bound`?

Encontrar o elemento imediatamente menor que x no set:

Quando usar `lower_bound` ou `upper_bound`?

Encontrar o elemento imediatamente menor que x no set:

```
auto it = s.lower_bound(x);  
if (it == s.begin())  
    cout << -1 << endl;  
else  
    cout << *(--s.lower_bound(x)) << endl;
```

Quando usar `lower_bound` ou `upper_bound`?

Encontrar quantos termos estão entre a e b :

Quando usar `lower_bound` ou `upper_bound`?

Encontrar quantos termos estão entre *a* e *b*:

```
auto l = lower_bound(v.begin(), v.end(), a);  
auto r = upper_bound(v.begin(), v.end(), b);  
cout << r - l << endl;
```


Exercícios Mentais

Há muitas aplicações para `lower_bound` e `upper_bound`. Pense em como resolvê-las:

- ▶ Quantos elementos são **iguais** a x no array?
- ▶ Quantos elementos são **maiores** que x no array?
- ▶ Quantos elementos são **positivos** no array?

Counting Haybales — Relembrando o Enunciado

Enunciado: O fazendeiro John possui N fardos de feno ($1 \leq N \leq 10^5$) em vários pontos ao longo de sua estrada. Ajude-o a responder Q perguntas ($1 \leq Q \leq 10^5$), cada uma solicitando o número de fardos dentro de um intervalo específico.

Entrada:

```
4 6
3 2 7 5
2 3
2 4
2 5
2 7
4 6
8 10
```

Saída:

```
2
2
3
4
1
0
```

Counting Haybales — Solução

Para cada consulta de intervalo $[a, b]$, deve-se calcular a distância entre:

- ▶ `lower_bound(v.begin(), v.end(), a)` — o primeiro elemento **incluso** no intervalo.
- ▶ `upper_bound(v.begin(), v.end(), b)` — o primeiro elemento **após** o intervalo.

A diferença entre esses dois iteradores é exatamente a quantidade de fardos no intervalo $[a, b]$.

Hora de Implementar

Counting Haybales

Implemente a solução utilizando `lower_bound` e `upper_bound`.

Accepted

Tempo: 0.00s — Memória: 0 KB

Nos vemos na próxima aula.